

Important SQL & PL/SQL Queries - DBMS Practical

1. Table Basics

```
CREATE TABLE student (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    marks INT  
);  
INSERT INTO student VALUES (1, 'Rushi', 85);  
SELECT * FROM student;  
UPDATE student SET marks = 90 WHERE id = 1;  
DELETE FROM student WHERE id = 1;
```

2. WHERE, LIKE, IN, BETWEEN

```
SELECT * FROM student WHERE marks > 80;  
SELECT * FROM student WHERE name LIKE 'R%';  
SELECT * FROM student WHERE marks BETWEEN 70 AND 90;  
SELECT * FROM student WHERE id IN (1, 3, 5);
```

3. Sorting and Limiting

```
SELECT * FROM student ORDER BY marks DESC;  
SELECT * FROM student ORDER BY name ASC LIMIT 1;
```

4. Aggregate Functions

```
SELECT COUNT(*) FROM student;  
SELECT AVG(marks), MAX(marks), MIN(marks) FROM student;  
SELECT name FROM student WHERE marks = (SELECT MAX(marks) FROM student);
```

5. GROUP BY and HAVING

```
SELECT grade, COUNT(*) FROM student GROUP BY grade;  
SELECT grade, COUNT(*) FROM student GROUP BY grade HAVING COUNT(*) > 2;
```

6. JOINS

```
SELECT s.name, c.course_name  
FROM student s  
JOIN course c ON s.course_id = c.course_id;  
  
SELECT s.name, c.course_name  
FROM student s  
LEFT JOIN course c ON s.course_id = c.course_id;
```

7. Views

Important SQL & PL/SQL Queries - DBMS Practical

```
CREATE VIEW high_scores AS SELECT * FROM student WHERE marks > 80;  
SELECT * FROM high_scores;
```

8. PL/SQL Block

```
DECLARE  
    total INT;  
BEGIN  
    SELECT SUM(marks) INTO total FROM student;  
    DBMS_OUTPUT.PUT_LINE('Total marks: ' || total);  
END;
```

9. Trigger (AFTER INSERT)

```
DELIMITER //  
CREATE TRIGGER trg_after_insert  
AFTER INSERT ON student  
FOR EACH ROW  
BEGIN  
    INSERT INTO log VALUES (NEW.id, NOW(), 'Inserted');  
END;  
//  
DELIMITER ;
```

10. Stored Procedure

```
DELIMITER //  
CREATE PROCEDURE update_marks(IN sid INT, IN extra INT)  
BEGIN  
    UPDATE student SET marks = marks + extra WHERE id = sid;  
END;  
//  
DELIMITER ;  
  
CALL update_marks(1, 5);
```